Relatório 8 - Laboratório de Introdução à Ciência da Computação II

Ryan Sulino, 16900070

September 2026

1 Implementação

1.1 Estratégia

A ideia principal é utilizar a busca binária no intervalo [0.01, M] para se encontrar o maior comprimento de cada um dos cabos após o corte, onde M é o maior cabo. Para isso, dividimos ao meio o maior cabo (junto com todos os outros na mesma região) e verificamos o número de cabos após isso, se o número de cabos após o corte for menor que a quantidade que queremos, realizamos o novo corte à esquerda, caso contrário, realizamos o novo corte à direita.

1.2 Complexidade Assintótica

A cada etapa da busca binária, temos que verificar a quantidade total de cabos, o que necessita $O(N)$ operações, e na busca em sí, dividimos iterativamente o intervalo [0.01, M] ao meio, necessitando $O(\log M)$.

Portanto, a complexidade total é $O(N \log M)$.

2 Soluções das IAs

2.1 Prompt e modelos

O prompt "Implemente em C esse exercício e faça a análise de complexidade. Essa etapa faz parte da atividade." foi enviado ao Claude Free e ao

Gemini Free.

2.2 Códigos

2.2.1 Claude

```
#include <stdio.h>
#include <stdlib.h>

/* Converte "123.45" para cent metros (123*100 + 45), sem usar float/double */
static long long parse_cm(const char *s) {
    long long inteiro = 0, frac = 0;
    int digitos = 0, depois_ponto = 0;

    for (; *s; s++) {
        if (*s == '.' || *s == ',') {
            depois_ponto = 1;
        } else if (*s >= '0' && *s <= '9') {
            if (!depois_ponto) {
                inteiro = inteiro * 10 + (*s - '0');
            } else if (digitos < 2) {
                frac = frac * 10 + (*s - '0');
                digitos++;
            }
        }
    }
    while (digitos < 2) { frac *= 10; digitos++; }
    return inteiro * 100 + frac;
}

/* Verifica se    poss vel obter pelo menos K cabos de comprimento x (cm) */
static int possivel(const long long *L, int n, long long k, long long x) {
    long long total = 0;
    for (int i = 0; i < n; i++) {
        total += L[i] / x;
        if (total >= k) return 1;
    }
    return 0;
}

int main(void) {
    int n;
    long long k;

    if (scanf("%d␣%lld", &n, &k) != 2) return 0;

    long long *L = (long long *)malloc(n * sizeof(long long));
    if (L == NULL) return 1;

    long long maior = 0;
    char buf[64];
    for (int i = 0; i < n; i++) {
        scanf("%63s", buf);
        L[i] = parse_cm(buf);
        if (L[i] > maior) maior = L[i];
```

```
    }

    long long lo = 1, hi = maior, resp = 0;
    while (lo <= hi) {
        long long mid = lo + (hi - lo) / 2;
        if (possivel(L, n, k, mid)) {
            resp = mid;
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }

    printf("%lld.%02lld\n", resp / 100, resp % 100);

    free(L);
    return 0;
}
```

2.2.2 Gemini

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int N;
    long long K;

    // Leitura de N e K
    if (scanf("%d␣%lld", &N, &K) != 2) return 0;

    // Aloca  o din mica para armazenar os comprimentos em cent metros
    long long *lengths = (long long *)malloc(N * sizeof(long long));
    if (lengths == NULL) {
        return 1;
    }

    long long max_len = 0;
    for (int i = 0; i < N; i++) {
        int m, c;
        scanf("%d.%d", &m, &c);
        // Converte metros para cent metros sem usar ponto flutuante
        lengths[i] = (long long)m * 100 + c;
        if (lengths[i] > max_len) {
            max_len = lengths[i];
        }
    }

    // Busca bin ria pelo maior comprimento poss vel
    long long low = 1;
    long long high = max_len;
    long long ans = 0;

    while (low <= high) {
        long long mid = low + (high - low) / 2;
        long long count = 0;
```

```
        // Conta quantos cabos de tamanho 'mid' podem ser cortados
        for (int i = 0; i < N; i++) {
            count += lengths[i] / mid;
        }

        if (count >= K) {
            ans = mid;       //    poss vel, guardamos a resposta e tentamos um valor maior
            low = mid + 1;
        } else {
            high = mid - 1;  // Cabos muito longos, precisamos diminuir o tamanho
        }
    }

    // Imprime o resultado em metros com exatamente duas casas decimais
    printf("%lld.%02lld\n", ans / 100, ans % 100);

    // Libera  o da mem ria alocada dinamicamente
    free(lengths);

    return 0;
}
```

2.3 Estratégias

2.3.1 Claude

Busca binária na resposta: se é possível obter K cabos de x cm, também é possível com qualquer comprimento menor. Essa monotonicidade permite buscar o maior x no intervalo [1, maior rolo]. A complexidade de tempo é $O(N \log M)$.

2.3.2 Gemini

Busca Binária: O intervalo de busca vai de 1 até M (onde M é o comprimento máximo do rolo em centímetros, com $M \leq 10^7$). a complexidade de tempo total é $O(N \log M)$.

3 Breve comparação

Tanto a implementação feita durante a aula quanto as dos modelos de linguagens utilizaram a mesma estratégia: busca binária. Todas as implementações e a análise de complexidade estavam corretas.

Quanto a legibilidade do código, o Claude forneceu uma implementação pouco legível, principalmente na parte de realizar o parsing do input do usuário, enquanto o Gemini forneceu uma solução similar à realizada durante a aula, com a diferença de um código mais condensado e sem utilizar funções.

4 Tabelas

Table 1: Tempo de execução para $N = 10$, variando K (Aluno).

K	Tempo (ns)
1	536
10	1263
10^2	1295
10^3	1268
10^4	1309
10^5	1316
10^6	1320
10^7	1327
10^8	1339
10^9	1329

Table 2: Tempo de execução para $N = 10$, variando K (Claude).

K	Tempo (ns)
1	264
10	1292
10^2	1277
10^3	1300
10^4	1276
10^5	1336
10^6	1296
10^7	1328
10^8	1320
10^9	1301

Table 3: Tempo de execução para $N = 10$, variando K (Gemini).

K	Tempo (ns)
1	132
10	1210
10^2	1283
10^3	1266
10^4	1268
10^5	1310
10^6	1273
10^7	1316
10^8	1308
10^9	1327

Table 4: Tempo de execução para $K = 10$, variando N (Aluno).

N	Tempo (ns)
10	1268
10^2	12478
10^3	112723
10^4	989696
10^5	8398420
$2 \cdot 10^5$	15794539

Table 5: Tempo de execução para $K = 10$, variando N (Claude).

N	Tempo (ns)
10	1316
10^2	13280
10^3	131816
10^4	1343093
10^5	13320646
$2 \cdot 10^5$	25739470

Table 6: Tempo de execução para $K = 10$, variando N (Gemini).

N	Tempo (ns)
10	1340
10^2	13281
10^3	128630
10^4	1285882
10^5	13082028
$2 \cdot 10^5$	26343205

5 Dificuldades e aprendizado com a comparação

A principal dificuldade foi compreender uma utilização não convencional da busca binária para a solução do problema.

Realizando a comparação do código com as LLMs foi possível perceber que todos os códigos eram otimizados em termos de eficiência e aplicaram corretamente o algoritmo, porém, deixaram a desejar em termos de legibilidade. Mesmo uma solução simples não foi efetuada da melhor maneira pelos modelos de linguagem.

Além disso, também é válido citar o nível de detalhes que as IAs fornecem na explicação das soluções.